Docker

DevOps Cheat Sheet · GCP Engineer Edition

IMAGE COMMANDS

`docker pull image:tag`	Download image from registry	*e.g. docker pull nginx:latest*
`docker build -t name:tag .`	Build image from Dockerfile in . folder	*Always tag with a version*
`docker images`	List all local images	
`docker rmi image:tag`	Delete a local image	*Free up disk space*
`docker image prune`	Remove all unused/dangling images	*Add -a to remove all unused*
`docker tag src:tag dst:tag`	Add a new name/tag to an image	*Used before pushing*
`docker push name:tag`	Upload image to a registry	*Login with docker login first*
`docker save -o file.tar img`	Export image to a tar file	*For air-gapped environments*

CONTAINER COMMANDS

`docker run image`	Create & start a container	
`docker run -d image`	Run container in background (detached)	*Most common for servers*
`docker run -p 8080:80 image`	Map host port 8080 → container port 80	*host:container*
`docker run -e KEY=val image`	Set an environment variable	*Pass secrets/config*
`docker run -v /h:/c image`	Mount host folder into container	*host_path:container_path*
`docker run --name myapp img`	Give container a custom name	*Easier than using IDs*
`docker run --rm image`	Auto-delete container when it stops	*Good for one-off tasks*
`docker ps`	List running containers	*Add -a to see stopped too*
`docker stop container`	Gracefully stop a container	*Sends SIGTERM then SIGKILL*
`docker rm container`	Delete a stopped container	*Add -f to force remove*
`docker logs container`	View container output/logs	*Add -f to follow live logs*
`docker exec -it cont bash`	Open a shell inside running container	*Great for debugging*
`docker inspect container`	Full JSON config of a container	*IPs, volumes, env vars*
`docker stats`	Live CPU/memory usage per container	
`docker cp file cont:/path`	Copy file into a running container	*Also works in reverse*

DOCKERFILE KEYWORDS

`FROM image:tag`	Base image to build on top of	*Always the first line*
`RUN command`	Execute a shell command at build time	*Installs packages, etc.*
`COPY src dest`	Copy files from host into image	*Use over ADD for local files*
`ADD src dest`	Like COPY but can untar and fetch URLs	
`WORKDIR /path`	Set working directory for following cmds	*Creates dir if missing*
`ENV KEY=value`	Set environment variable in image	*Available at runtime too*
`ARG NAME`	Build-time variable (not in final image)	*Pass with --build-arg*
`EXPOSE port`	Document which port the app listens on	*Does NOT publish the port*
`CMD ["app","arg"]`	Default command to run when container starts	*Can be overridden at runtime*
`ENTRYPOINT ["app"]`	Executable that always runs	*CMD becomes its arguments*
`VOLUME ["/data"]`	Declare a mount point	
`USER name`	Switch to a non-root user	*Security best practice*
`HEALTHCHECK CMD …`	Command Docker uses to check health	
`LABEL key=value`	Add metadata to image	*e.g. version, maintainer*

DOCKER COMPOSE & GOOGLE CONTAINER REGISTRY (GCR/AR)

`docker compose up -d`	Start all services in background	*Reads docker-compose.yml*
`docker compose down`	Stop and remove containers + networks	*Add -v to also remove volumes*
`docker compose logs -f svc`	Follow logs of one service	
`docker compose build`	Rebuild images for all services	
`docker compose ps`	List service containers and status	
`gcloud auth configure-docker`	Authenticate Docker to GCR/AR	*Run once per machine*
`docker tag img gcr.io/proj/img`	Tag image for Google Container Registry	
`docker push gcr.io/proj/img`	Push to Google Container Registry	
`docker pull ar.io/proj/img`	Pull from Artifact Registry	*New GCP registry*

MINIMAL DOCKERFILE EXAMPLE (Node.js app)

```
FROM node:20-alpine WORKDIR /app # Install dependencies first (better layer caching) COPY package*.json ./ RUN
npm ci --only=production # Copy application code COPY . . # Security: run as non-root USER node EXPOSE 3000 CMD
["node", "server.js"]
```

Tip: Use .dockerignore to exclude node_modules, .git, secrets from your image build. **docs.docker.com**